

Task 03 — Database-per-Service & Data Ownership

Step 1 — Learn the concepts

The first step in your Jira task asks you to understand these four concepts.

Shared Database

Multiple services use the same database.

User Service ----\
Order Service -----> One Database
Payment Service --/

This is generally **not preferred** in microservices because services become tightly coupled.

Database-per-Service

Each service has its own database.

User Service -----> User DB
Order Service -----> Order DB
Payment Service ---> Payment DB

This is the main concept your task wants you to implement.

Schema-per-Service

Instead of completely separate database servers/databases, you can have separate schemas.

Database

|— user_schema
 └─ users
|— order_schema
 └─ orders
 └─ order_items
└─ payment_schema
 └─ payments

The important point is still **ownership**: each service owns its schema/data.

Data Ownership

User Service

owns

users table

Only User Service should modify user information.

Order Service should **not** directly run:

```
UPDATE UserDB.dbo.users →SQL
```

Instead, Order Service should communicate with User Service through an API.

Step 2 — Create three services

your task appears to expect Java-based services.

structure could be:

MicroservicesSolution

|

|— UserService

|

|— OrderService

|

|— PaymentService

If your team has already provided a specific Java/Spring Boot structure, follow that structure.

Step 3 — Each service its own database/schema

This is one of the most important parts.

User Service

Create:

users

Example:

users

id

name

email

password

created_at

The **User Service** owns this table.

Order Service

Create:

orders

order_items

Example:

orders

id

user_id

order_date

status

total_amount

and:

order_items

id

order_id

product_id

quantity

price

The **Order Service** owns both tables.

Payment Service

Create:

payments

Example:

payments

id

order_id

amount

payment_status

payment_date

The **Payment Service** owns this table.

Step 4 — Create entities/tables

create the database tables and corresponding Java entities.

Example:

User Service

|

+-- User.java

|

+-- users table

Order Service

|

+--- Order.java

+--- OrderItem.java

|

+--- orders table

+--- order_items table

Payment Service

|

+--- Payment.java

|

+--- payments table

So your Java entities represent the tables owned by that particular service.

Step 5

Do not directly access another service's database.

Order Service

|

| HTTP/API

v

User Service

|

v

User Database

Example: GET /api/users/10

User Service accesses its own database and returns the information.

Step 6 — Design APIs

APIs through which services communicate.

Example:

User Service

GET /api/users/{id}

POST /api/users

PUT /api/users/{id}

DELETE /api/users/{id}

Order Service

GET /api/orders/{id}

POST /api/orders

Payment Service

GET /api/payments/{id}

POST /api/payments

Step 7 — Document ownership

Who owns each database?

create something like:

Service	Database/Schema Tables		Owner
User Service	User DB	users	User Service
Order Service	Order DB	orders, order_items	Order Service
Payment Service	Payment DB	payments	Payment Service

Who can modify the data?

Example:

User Service



Can INSERT/UPDATE/DELETE users

Order Service



Can INSERT/UPDATE/DELETE orders/order_items

Payment Service



Can INSERT/UPDATE/DELETE payments

Other services cannot directly modify those tables.

How do services exchange information?

REST API

or potentially:

Message Broker

Example:

Order Service



| GET /api/users/10



User Service



User DB

Step 8 — Test the architecture

Example:

Test 1

Create a user:

POST /api/users

User gets stored in:

User DB



users

Test 2

Create an order:

POST /api/orders

Order gets stored in:

Order DB



orders

Test 3

Order Service needs user information.

It calls:

User Service API

It does **not** access User DB directly.

Step 9 — Document common problems

1. Duplicate Data

Sometimes the same information may exist in multiple services.

Example:

User Service

users

user_id

name

email

Order Service might need to store:

orders

order_id

user_id

You shouldn't blindly copy the entire user record into Order DB.

Data Synchronization

Suppose a user changes their email:

old@email.com

↓

new@email.com

If another service stores a copy of the email, you need to consider how that copy gets updated.

This is a common challenge in distributed systems.

Distributed Transactions

Suppose:

Create Order

+

Make Payment

These are owned by different services/databases.

Order DB Payment DB

| |
V v

Order Service Payment Service

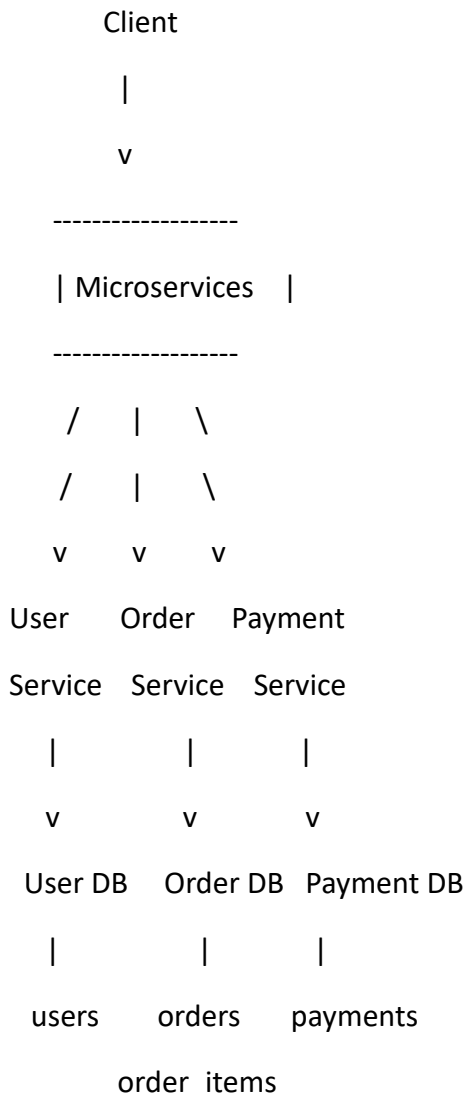
You cannot simply treat both databases like one normal database transaction.

This is called a **distributed transaction problem**.

Final Deliverable

Database Architecture

Create a diagram like:



ER Diagrams

Create ER diagrams for:

User Service

USERS

id PK

name

email

password

Order Service

ORDERS

id PK

user_id

order_date

status

|

| 1-to-many

v

ORDER_ITEMS

id PK

order_id FK

product_id

quantity

price

Payment Service

PAYMENTS

id PK

order_id

amount

status

payment_date

Notice something important:

user_id in Order DB is **not a database foreign key to User DB**.

It's simply an identifier/reference.

Database Configuration

Need to configure each service with its own database connection.

User Service



User DB

Order Service



Order DB

Payment Service



Payment DB

Not:

All services



One DB

Java Entities

Need to provide entities

User.java

Order.java

OrderItem.java

Payment.java

These belong to their respective services.

API/Data Ownership Documentation

User Service

Owns: users

Provides: User APIs

Order Service

Owns: orders, order_items

Provides: Order APIs

Payment Service

Owns: payments

Provides: Payment APIs

Test Evidence

Finally, show evidence that tested it.

POST /api/users

↓

201 Created

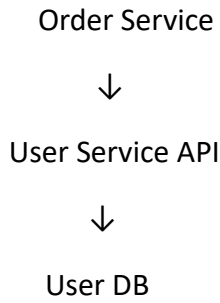
and:

POST /api/orders

↓

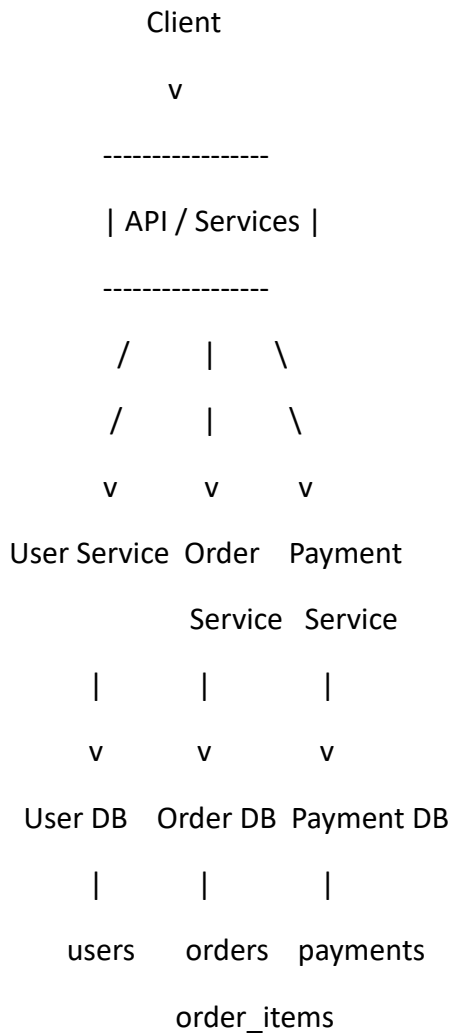
201 Created

and:



No service directly connects to another service's database.

Your final architecture



Service owns its data → Service is the only one that directly accesses/modifies that data →
Other services communicate through APIs/events, not by directly accessing the database.